

Codora Technical Architecture

Current Direction

Use `Next.js` for the frontend and backend API. Use `Supabase` for auth and database. Use GitHub login.

Required Environment

The app will need environment configuration for:

- database connection
- Supabase URL
- Supabase publishable key
- Supabase secret key, if any server-side operations require it
- GitHub OAuth setup through Supabase
- any future execution or grading service keys

High-Level Shape

```
flowchart TD
    U[User] --> UI[Next.js App]
    UI --> DB[(Supabase)]
    UI --> ATT[Attempts]
    UI --> LIB[Problems / Reviewer Content]
    UI --> STATS[Progress / Leaderboards]
```

Responsibilities

Next.js

- Render the product UI
- Handle onboarding screens
- Show the problem library
- Host the code editor experience
- Expose backend API routes for submissions, attempts, progress, and content access
- Capture submissions and attempts
- Display stats and leaderboards
- Show reviewer/docs pages

Supabase

- Handle authentication
- Support GitHub login
- Store users and profiles

- Store problems and reviewer content
- Store attempts and submission history
- Store tags, topics, and difficulty metadata
- Store progress and leaderboard data

Core Data Boundaries

User Profile

Holds:

- proficiency level
- learning tags
- profile metadata

Problem

Holds:

- title
- prompt
- topic
- difficulty
- starter code
- visible example
- hidden test cases
- explanation/review content

Attempt

Holds:

- user
- problem
- submitted code
- timestamps
- timer data
- pass/fail result
- test evaluation result

Progress

Derived from attempts and problem outcomes.

Product Rules To Preserve In The Architecture

- Attempts must be stored.
- An attempt passes only when all tests pass.
- The first supported language is Python.
- The reviewer area is read-only.
- The problem library and reviewer library should both support topic and difficulty browsing.
- Progress and leaderboard views should come from the same attempt history.
- Login should use GitHub through Supabase Auth.

Content Pipeline

The app should support later AI generation of problems and test cases through a separate script that writes into the database. That content pipeline is distinct from the user-facing solving flow.

Open Build Boundary

The current stack direction is intentionally small. The detailed runtime for executing and grading submitted Python code still needs a dedicated implementation path, but it should not change the product shape already defined here.

Risks

- Grading must stay trustworthy.
- Progress stats should not feel fake or inflated.
- The reviewer area should not blur into a teaching workflow.
- Leaderboard scoring must feel competitive without obscuring correctness.